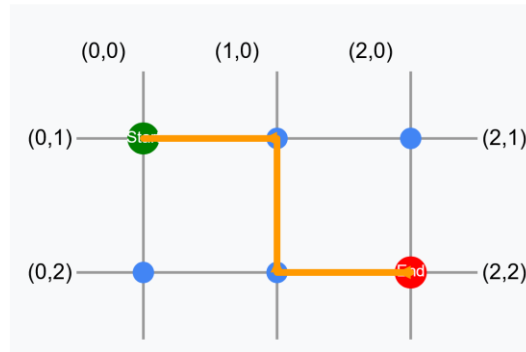


5th EXERCISE: GRID PATHS - COUNTING ROUTES IN A GRID

THE PROBLEM (in simple terms)

Imagine you're standing at the top-left corner of a grid of streets. You want to reach the bottom-right corner, but you can only move in two directions: right or down. **How many different routes can you take?**

For example, in a 2×2 grid (3×3 intersection points), there are 6 possible paths from the top-left to the bottom-right corner.



REAL-LIFE EXAMPLE

Think of this like navigating through a city with streets laid out in a perfect grid (like parts of Manhattan). You're at the northwest corner and want to reach the southeast corner. You can only travel east or south along the streets. How many different routes can you take?

This problem has real applications in:

- **Robotics:** Planning how robots move
- **Delivery routing:** Finding possible delivery paths
- **Network design:** Planning cable or pipeline layouts
- **Games:** Calculating possible moves in grid-based games

UNDERSTANDING THE MATHS (don't worry, we'll keep it simple!)

To solve this problem, we need to think about the total number of moves we must make:

To get from the top-left to the bottom-right of an $m \times n$ grid, we need:

- **m** moves to the right
- **n** moves down
- Total of **$m+n$** moves

The key insight: Every path must contain exactly **m** right moves and **n** down moves. The only thing that changes is the order of these moves.

So, this becomes a question of: "In how many ways can we arrange **m** right moves and **n** down moves?"

This is a combination problem, and mathematicians write it as:

$$\text{Number of paths} = (m+n)! / (m! \times n!)$$

Don't worry if that looks confusing! We'll approach it step by step with code.

SOLVING WITH RECURSION (breaking it down)

Let's think about this problem differently:

To reach any point (x,y) in the grid, you can only come from:

1. The cell directly above (x, y-1),

or

2. The cell directly to the left (x-1, y)

So the number of ways to reach (x,y) is:

- The number of ways to reach the cell above,

PLUS

- The number of ways to reach the cell to the left

We can write this as a simple rule:

```
paths(x, y) = paths(x-1, y) + paths(x, y-1)
```

But what about the edges of the grid? If you're on the top edge or left edge, there's only ONE way to reach any point (just go straight along the edge).

LET'S CODE IT! (PYTHON VERSION)

```
def grid_paths(m, n, memo=None):
    # First time calling? Create our memory storage
    if memo is None:
        memo = {}

    # Base case: if we're at the top or left edge, there's only one path
    if m == 0 or n == 0:
        return 1

    # Have we solved this before? If yes, use the saved result
    if (m, n) in memo:
        return memo[(m, n)]

    # Otherwise, calculate and save the result
    memo[(m, n)] = grid_paths(m - 1, n, memo) + grid_paths(m, n - 1, memo)
    return memo[(m, n)]

# Try it with a 2x2 grid
print(grid_paths(2, 2)) # Output: 6
```

Remainder: this code uses a technique called "memoisation" (storing results we've already calculated). Without this, the code would be very slow for larger grids.

C VERSION (for those interested)

```
#include <stdio.h>

#define MAX 100

int memo[MAX][MAX] = {0};

int grid_paths(int m, int n) {
    // Base case: if we're at the top or left edge, there's only one path
    if (m == 0 || n == 0)
        return 1;

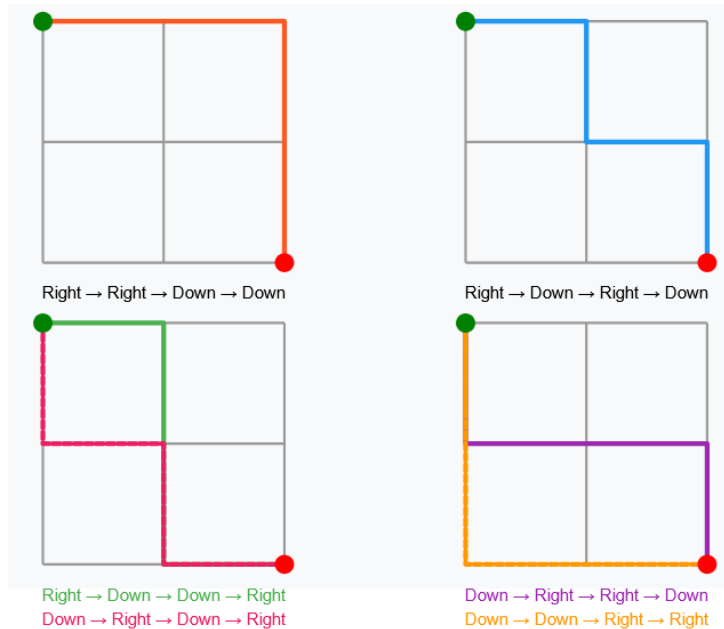
    // If we've already calculated this value, return it
    if (memo[m][n] != 0)
        return memo[m][n];

    // Otherwise, calculate and save the result
    memo[m][n] = grid_paths(m - 1, n) + grid_paths(m, n - 1);
    return memo[m][n];
}
```

```
int main() {
    printf("%d\n", grid_paths(2, 2)); // Output: 6
    return 0;
}
```

STEP-BY-STEP EXAMPLE: A 2×2 GRID

Let's trace how this works for a 2×2 grid, starting at (0,0) and heading to (2,2):



1. Call `grid_paths(2, 2)`
 - We need to find `grid_paths(1, 2) + grid_paths(2, 1)`
2. Calculate `grid_paths(1, 2)`:
 - We need to find `grid_paths(0, 2) + grid_paths(1, 1)`
 - `grid_paths(0, 2) = 1` (base case: left edge)
 - So we need `1 + grid_paths(1, 1)`
3. Calculate `grid_paths(1, 1)`:
 - We need to find `grid_paths(0, 1) + grid_paths(1, 0)`
 - `grid_paths(0, 1) = 1` (base case: left edge)
 - `grid_paths(1, 0) = 1` (base case: top edge)
 - So `grid_paths(1, 1) = 1 + 1 = 2`
4. Now we can finish `grid_paths(1, 2)`:
 - `grid_paths(1, 2) = 1 + 2 = 3`
5. Calculate `grid_paths(2, 1)`:
 - We need to find `grid_paths(1, 1) + grid_paths(2, 0)`
 - We already know `grid_paths(1, 1) = 2`
 - `grid_paths(2, 0) = 1` (base case: top edge)
 - So `grid_paths(2, 1) = 2 + 1 = 3`
6. Finally, we can complete `grid_paths(2, 2)`:
 - `grid_paths(2, 2) = 3 + 3 = 6`

So there are **6** possible paths in a 2×2 grid!

ALL POSSIBLE PATHS IN A 2×2 GRID

Here are all six paths from (0,0) to (2,2):

1. Right → Right → Down → Down
2. Right → Down → Right → Down
3. Right → Down → Down → Right
4. Down → Right → Right → Down
5. Down → Right → Down → Right
6. Down → Down → Right → Right

Notice that each path contains exactly 2 right moves and 2 down moves.

WHY MEMOISATION IS IMPORTANT

Without storing our results (memoisation), we would recalculate the same values many times. For example, in computing `grid_paths(3, 3)`, we would calculate `grid_paths(1, 1)` multiple times.

With memoisation, we calculate it once and reuse that value whenever needed, making our code much faster.

A MORE EFFICIENT APPROACH: BUILDING A TABLE

Instead of using recursion, we can build a table of values:

```
def grid_paths_table(m, n):
    # Create a table of size (m+1) x (n+1) filled with zeros
    table = [[0 for _ in range(n+1)] for _ in range(m+1)]

    # Fill in the edges with 1s (there's only one way to reach any point on the edge)
    for i in range(m+1):
        table[i][0] = 1
    for j in range(n+1):
        table[0][j] = 1

    # Fill in the rest of the table
    for i in range(1, m+1):
        for j in range(1, n+1):
            table[i][j] = table[i-1][j] + table[i][j-1]

    # Return the bottom-right value
    return table[m][n]
```

This method is called "*dynamic programming with tabulation*" and is often more efficient than the recursive approach.

USING THE MATHEMATICAL FORMULA

For those comfortable with maths, we can use the combination formula directly:

```
def grid_paths_math(m, n):
    from math import factorial
    return factorial(m + n) // (factorial(m) * factorial(n))
```

This is the fastest method but might have issues with very large grids due to factorial calculations potentially causing numerical overflow.

HOW FAST ARE THESE METHODS?

1. **Simple Recursion (without memoisation):**
 - Very slow for larger grids: $O(2^{m+n})$ time complexity
 - This means the time doubles (or worse) with each increase in grid size
2. **Recursion with Memoisation:**
 - Much better: $O(m \times n)$ time complexity
 - We only calculate each grid point once

3. Table Method (Dynamic Programming):

- Also $O(m \times n)$ time complexity
- Often slightly faster in practice than the memoised recursion

4. Mathematical Formula:

- Even faster: $O(m+n)$ time complexity
- But may have issues with very large numbers

EXTENSION IDEAS: MAKE IT MORE INTERESTING!

Once you understand the basic problem, you can try these extensions:

1. **Blocked Paths:** What if some cells are blocked and you can't go through them?
2. **Weighted Paths:** What if some moves cost more than others? Find the cheapest path!
3. **Diagonal Moves:** What if you can also move diagonally?
4. **Listing All Paths:** Instead of just counting paths, can you list all possible paths?

WHAT WE'VE LEARNED

This problem teaches us several important concepts:

1. **Breaking down problems:** Solving complex problems by breaking them into simpler subproblems
2. **Memoisation:** Storing results to avoid repeating calculations
3. **Dynamic Programming:** Building solutions from the bottom up
4. **Mathematical Connections:** Seeing how code relates to mathematical formulas
5. **Efficiency trade-offs:** Understanding how different approaches affect speed and memory usage

The grid paths problem is a perfect introduction to these concepts because it's easy to understand but demonstrates powerful programming techniques.